4
Introducing Convolutional Neural Networks

So far, we've learned how to build deep neural networks and the impact of tweaking their various hyperparameters. In this chapter, we will learn about where traditional deep neural networks do not work. We'll then learn about the inner workings of **convolutional neural networks (CNNs)** by using a toy example before understanding some of their major hyperparameters, including stride, pooling, and filters. Next, we will leverage CNNs, along with various data augmentation techniques, to solve the issue of traditional deep neural networks not having good accuracy. Following this, we will learn what the outcome of a feature learning process in a CNN looks like. Finally, we'll bring our learning together to solve a use case: we'll be classifying an image by stating whether the image contains a dog or a cat. By doing this, we'll be able to understand how the accuracy of prediction varies by the amount of data available for training.

By the end of this chapter, you will have a deep understanding of CNNs, which form the backbone of multiple model architectures that are used for various tasks.

The following topics will be covered in this chapter:

- The problem with traditional deep neural networks
- Building blocks of a CNN
- Implementing a CNN
- Classifying images using deep CNNs
- Implementing data augmentation
- Visualizing the outcome of feature learning
- Building a CNN for classifying real-world images

Let's get started!

All code in this chapter is available for reference in the `Chapter04` folder of this book's GitHub repository: **https://bit.ly/mcvp-2e**.

The problem with traditional deep neural networks

Before we dive into CNNs, let's look at the major problem that's faced when using traditional deep neural networks.

Let's reconsider the model we built on the Fashion-MNIST dataset in *Chapter 3*. We will fetch a random image and predict the class that corresponds to that image, as follows:

The following code can be found in the `Issues_with_image_translation.ipynb` file located in the `Chapter04` folder on GitHub at **https://bit.ly/mcvp-2e**. Only the additional code corresponding to the issue of image translation will be discussed here for brevity.

1. Fetch a random image from the available training images:

```
# Note that you should run the code in
# Batch size of 32 section in Chapter 3
# before running the following code
import matplotlib.pyplot as plt
%matplotlib inline
# ix = np.random.randint(len(tr_images))
ix = 24300
plt.imshow(tr_images[ix], cmap='gray')
plt.title(fmnist.classes[tr_targets[ix]])
```

The preceding code results in the following output:

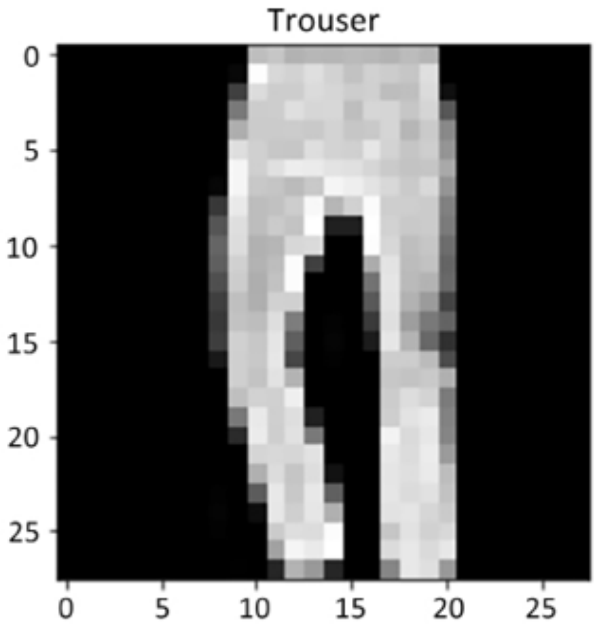


Figure 4.1: Image corresponding to index 24300

2. Pass the image through the **trained model** (continue using the model we trained in the *Batch size of 32* section of *Chapter 3*):
    1. Preprocess the image so it goes through the same preprocessing steps we performed while building the model:

```
img = tr_images[ix]/255.
img = img.view(28*28)
img = img.to(device)
```

    2. Extract the probabilities associated with the various classes:

```
np_output = model(img).cpu().detach().numpy()
np.exp(np_output)/np.sum(np.exp(np_output))
```

The preceding code results in the following output, where we can see that the highest probability is for the first index, which is of the `Trouser` class:

```
array([1.7608897e-08, 1.0000000e+00, 2.6042574e-13, 1.1353759e-10,
       3.1050048e-12, 7.2957764e-16, 8.0109371e-11, 3.8039204e-22,
       1.2800090e-15, 2.8759430e-18], dtype=float32)
```

Figure 4.2: Probabilities of different classes

3. Translate (roll/slide) the image multiple times (one pixel at a time) from a translation of 5 pixels to the left to 5 pixels to the right and store the predictions in a list:
    1. Create a list that stores predictions:

```
preds = []
```

    2. Create a loop that translates (rolls) an image from -5 pixels (5 pixels to the left) to +5 pixels (5 pixels to the right) of the original position (which is at the center of the image):

```
for px in range(-5,6):
```

    In the preceding code, we specified 6 as the upper bound even though we are interested in translating until +5 pixels since the output of the range would be from -5 to +5 when (-5,6) is the specified range.
    3. Preprocess the image as we did in *step 2*:

```
    img = tr_images[ix]/255.
    img = img.view(28, 28)
```

    4. Roll the image by a value equal to `px` within the `for` loop:

```
    img2 = np.roll(img, px, axis=1)
```

    Here, we specified `axis=1` since we want the image pixels to be moving horizontally and not vertically.
    5. Store the rolled image as a tensor object and register it to `device`:

```
    img3 = torch.Tensor(img2).view(28*28).to(device)
```

6. Pass `img3` through the trained model to predict the class of the translated (rolled) image and append it to the list that stores predictions for various translations:

```
    np_output = model(img3).cpu().detach().numpy()
    preds.append(np.exp(np_output)/np.sum(np.exp(np_output)))
```

4. Visualize the predictions of the model for all the translations (-5 pixels to +5 pixels):

```
import seaborn as sns
fig, ax = plt.subplots(1,1, figsize=(12,10))
plt.title('Probability of each class \
for various translations')
sns.heatmap(np.array(preds), annot=True, ax=ax, fmt='.2f',
xticklabels=fmnist.classes,yticklabels=[str(i)+ \
str(' pixels') for i in range(-5,6)], cmap='gray')
```

The preceding code results in the following output:

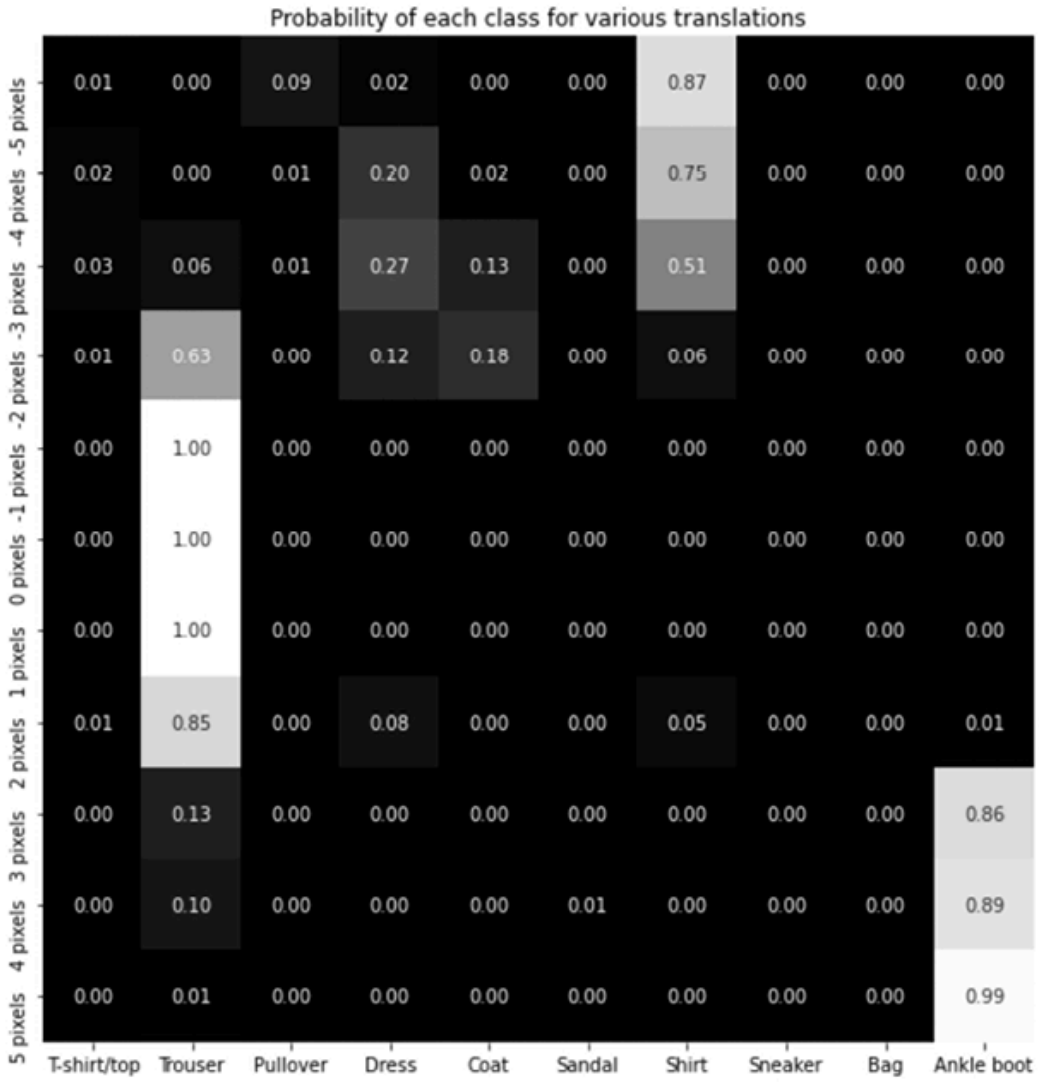


Figure 4.3: Probability of each class for different translations

There was no change in the image's content since we only translated the images from 5 pixels to the left and 5 pixels to the right. However, the predicted class of the image changed when the translation was beyond 2 pixels. This is because, while the model was being trained, the content in all the training and testing images was at the center. This differs from the preceding scenario where we tested with translated images that are off-center (by a margin of 5 pixels), resulting in an incorrectly predicted class.

Now that we have learned about a scenario where a traditional neural network fails, we will learn how CNNs help address this problem. But before we do this, let's first look at the building blocks of a CNN.

Building blocks of a CNN

CNNs are the most prominent architectures that are used when working on images. They address the major limitations of deep neural networks, like the one we saw in the previous section. Besides image classification, they also help with object detection, image segmentation, GANs, and much more – essentially, wherever we use images. Furthermore, there are different ways of constructing a CNN, and there are multiple pre-trained models that leverage CNNs to perform various tasks. Starting with this chapter, we will be using CNNs extensively.

In the upcoming subsections, we will understand the fundamental building blocks of a CNN, which are as follows:

- Convolutions
- Filters
- Strides and padding
- Pooling

Let's get started!

Convolution

A convolution is basically a multiplication between two matrices. As you saw in the previous chapter, matrix multiplication is a key ingredient in training a neural network. (We perform matrix multiplication when we calculate hidden layer values – which is a matrix multiplication of the input values and weight values connecting the input to the hidden layer. Similarly, we perform matrix multiplication to calculate output layer values.)

To ensure that we have a solid understanding of the convolution process, let's go through an example. Let's assume we have two matrices we can use to perform convolution.

Here is Matrix A:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Figure 4.4: Matrix A

Here is Matrix B:

1	2
3	4

Figure 4.5: Matrix B

While performing the convolution operation, we are sliding Matrix B (the smaller matrix) over Matrix A (the bigger matrix). Furthermore, we are performing element-to-element multiplication between Matrix A and Matrix B, as follows:

1. Multiply {1,2,5,6} of the bigger matrix by {1,2,3,4} of the smaller matrix:

*1*1 + 2*2 + 5*3 + 6*4 = 44*

2. Multiply {2,3,6,7} of the bigger matrix by {1,2,3,4} of the smaller matrix:

*2*1 + 3*2 + 6*3 + 7*4 = 54*

3. Multiply {3,4,7,8} of the bigger matrix by {1,2,3,4} of the smaller matrix:

*3*1 + 4*2 + 7*3 + 8*4 = 64*

4. Multiply {5,6,9,10} of the bigger matrix by {1,2,3,4} of the smaller matrix:

*5*1 + 6*2 + 9*3 + 10*4 = 84*

5. Multiply {6,7,10,11} of the bigger matrix by {1,2,3,4} of the smaller matrix:

*6*1 + 7*2 + 10*3 + 11*4 = 94*

6. Multiply {7,8,11,12} of the bigger matrix by {1,2,3,4} of the smaller matrix:

*7*1 + 8*2 + 11*3 + 12*4 = 104*

7. Multiply {9,10,13,14} of the bigger matrix by {1,2,3,4} of the smaller matrix:

*9*1 + 10*2 + 13*3 + 14*4 = 124*

8. Multiply {10,11,14,15} of the bigger matrix by {1,2,3,4} of the smaller matrix:

*10*1 + 11*2 + 14*3 + 15*4 = 134*

9. Multiply {11,12,15,16} of the bigger matrix by {1,2,3,4} of the smaller matrix:

*11*1 + 12*2 + 15*3 + 16*4 = 144*

The result of performing the preceding operations is as follows:

44	54	64
84	94	104
124	134	144

Figure 4.6: Output of convolution operation

The smaller matrix is typically called a **filter** or a kernel, while the bigger matrix is the original image.

Filters

A filter is a matrix of weights that is initialized randomly at the start. The model learns the optimal weight values of a filter over increasing epochs. The concept of filters brings us to two different aspects:

- What the filters learn about
- How filters are represented

In general, the more filters there are in a CNN, the more features of an image the model can learn about. We will learn about what various filters learn in the *Visualizing the outcome of feature learning* section of this chapter. For now, we'll proceed with the intermediate understanding that the filters learn about different features present in the image. For example, a certain filter might learn about the ears of a cat and provide high activation (a matrix multiplication value) when the part of the image it is convolving with contains the ear of a cat.

In the previous section, when we convolved one filter that has a size of 2 x 2 with a matrix that has a size of 4 x 4, we got an output that is 3 x 3 in dimension. However, if 10 different filters multiply the bigger matrix (original image), the result is 10 sets of the 3 x 3 output matrices.

In the preceding case, a 4 x 4 image is convolved with 10 filters that are 2 x 2 in size, resulting in 3 x 3 x 10 output values. Essentially, when an image is convolved by multiple filters, the output has as many channels as there are filters that the image is convolved with.

Furthermore, in a scenario where we are dealing with color images where there are three channels, the filter that is convolving with the original image would also have three channels, resulting in a single scalar output per convolution. Also, if the filters are convolving with an intermediate output – let's say 64 x 112 x 112 in shape – the filter would have

64 channels to fetch a scalar output. In addition, if there are 512 filters that are convolving with the output that was obtained in the intermediate layer, the output post-convolution with 512 filters would be 512 x 112 x 112 in shape.

To solidify our understanding of the output of filters further, let's take a look at the following diagram:

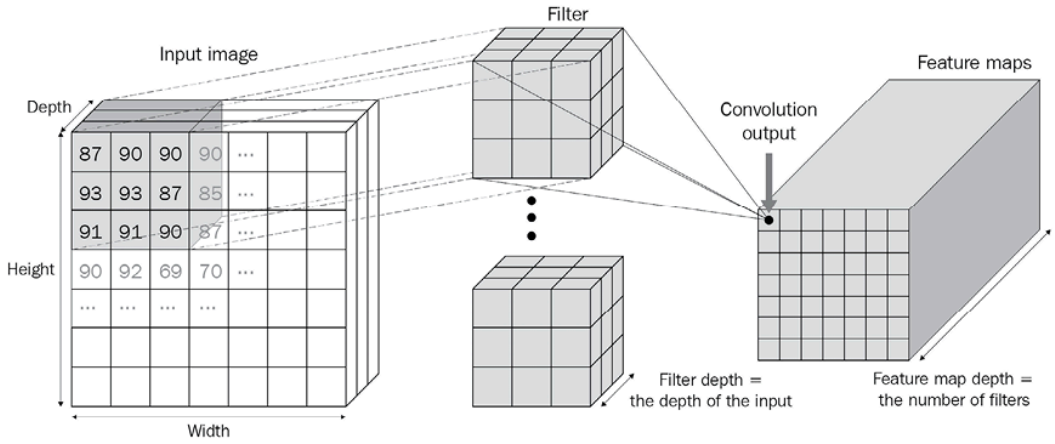


Figure 4.7: Output of convolution operation with multiple filters

In the preceding diagram, we can see that the input image is multiplied by the filters that have the same depth as that of the input (which the filters are convolving with) and that the number of channels in the output of a convolution is as many as there are filters.

Strides and padding

In the previous section, each filter strode across the image – one column and one row at a time (after exhausting all possible columns by the end of the image). This also resulted in the output size being 1 pixel less than the input image size – both in terms of height and width. This results in a partial loss of information and can limit the possibility of us adding the output of the convolution operation to the original image if the output of convolution and the original image are not of the same shape. This is known as residual addition and will be discussed in detail in the next chapter. For now, however, let's learn how strides and padding influence the output shape of convolutions.

Strides

Let's understand the impact of strides by leveraging the same example we saw in the *Filters* section. We'll move Matrix B with a stride of 2 over Matrix A. As a result, the output of convolution with a stride of 2 is as follows:

1. {1,2,5,6} of the bigger matrix is multiplied by {1,2,3,4} of the smaller matrix:

*1*1 + 2*2 + 5*3 + 6*4 = 44*

2. {3,4,7,8} of the bigger matrix is multiplied by {1,2,3,4} of the smaller matrix:

*3*1 + 4*2 + 7*3 + 8*4 = 64*

3. {9,10,13,14} of the bigger matrix is multiplied by {1,2,3,4} of the smaller matrix:

*9*1 + 10*2 + 13*3 + 14*4 = 124*

4. {11,12,15,16} of the bigger matrix is multiplied by {1,2,3,4} of the smaller matrix:

*11*1 + 12*2 + 15*3 + 16*4 = 144*

The result of performing the preceding operations is as follows:

44	64
124	144

Figure 4.8: Output of convolution with stride

Since we now have a stride of 2, note that the preceding output has a lower dimension compared to the scenario where the stride was 1 (where the output shape was 3 x 3).

Padding

In the preceding case, we could not multiply the leftmost elements of the filter by the rightmost elements of the image. If we were to perform such matrix multiplication, we would pad the image with zeros. This would ensure that we can perform element-to-element multiplication of all the elements within an image with a filter.

Let's understand padding by using the same example we used in the *Convolution* section. Once we add padding on top of Matrix A, the revised version of Matrix A will look as follows:

0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

Figure 4.9: Padding on Matrix A

As you can see, we have padded Matrix A with zeros, and the convolution with Matrix B will not result in the output dimension being smaller than the input dimension. This aspect comes in handy when we are working on a residual network where we must add the output of the convolution to the original image.

Once we've done this, we can perform activation on top of the convolution operation's output. We could use any of the activation functions we saw in *Chapter 3* for this.

Pooling

Pooling aggregates information in a small patch. Imagine a scenario where the output of convolution activation is as follows:

1	2
3	4

Figure 4.10: Output of convolution operation

The max pooling for this patch is 4 – as that is the maximum value across the values in the patch. Let's understand the max pooling for a bigger matrix:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Figure 4.11: Output of convolution operation

In the preceding case, if the pooling stride has a length of 2, the max pooling operation is calculated as follows, where we divide the input image by a stride of 2 (that is, we have divided the image into 2 x 2 divisions):

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Figure 4.12: Output of convolution with stride highlighted

For the four sub-portions of the matrix, the maximum values in the pool of elements are as follows:

6	8
14	16

Figure 4.13: Max pool values

In practice, it is not necessary to always have a stride of 2; this has just been used for illustration purposes here. Other variants of pooling are sum and average pooling. However, in practice, max pooling is used more often.

Note that by the end of performing the convolution and pooling operations, the size of the original matrix is reduced from 4 x 4 to 2 x 2. In a realistic scenario, if the original image is of shape 200 x 200 and the filter is of shape 3 x 3, the output of the convolution operation would be 198 x 198. Following that, the output of the pooling operation with a stride of 2 is 99 x 99. This way, by leveraging pooling, we preserve the more important features while reducing the dimension of inputs.

Putting them all together

So far, we have learned about convolution, filters, strides, padding, and pooling, and their impact on reducing the dimension of an image. Now, we will learn about another critical component of a CNN – the flatten layer (fully connected layer) – before putting the three pieces we have learned about together.

To understand the flattening process, we'll take the output of the pooling layer in the previous section and flatten the output. The output of flattening the pooling layer is {6, 8, 14, 16}.

By doing this, we'll see that the flatten layer can be treated as equivalent to the input layer (where we flattened the input image into a 784-dimensional input in *Chapter 3*). Once the flatten layer's (fully connected layer) values have been obtained, we can pass it through the hidden layer and then obtain the output for predicting the class of an image.

The overall flow of a CNN is as follows:

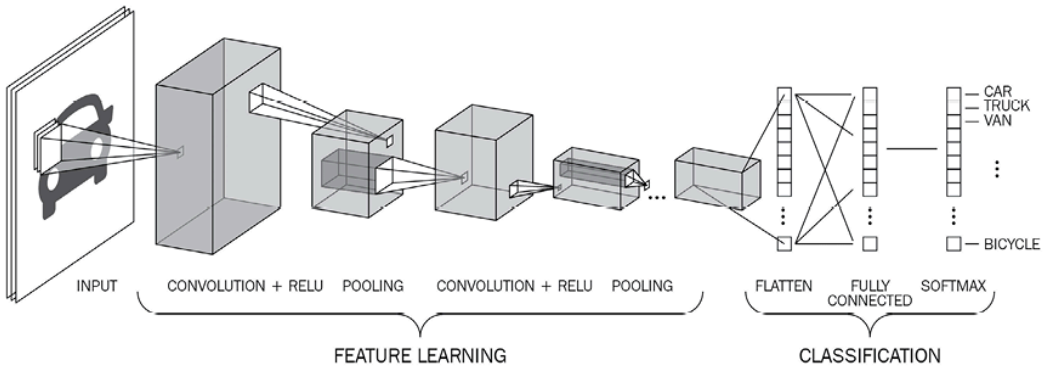


Figure 4.14: CNN workflow

We can see the overall flow of a CNN model, where we are passing an image through convolution via multiple filters and then pooling (and in the preceding case, repeating the convolution and pooling process twice), before flattening the output of the final pooling layer. This forms the **feature learning** part of the preceding image, where we are taking an image and reducing it to a lower dimension (the flattened output) while restoring the required information.

The operations of convolution and pooling constitute the feature learning section, as filters help in extracting relevant features from images and pooling helps in aggregating information and thereby reducing the number of nodes at the flatten layer.

If we directly flatten the input image (which is 300 x 300 pixels in size, for example), we are dealing with 90K input values. If we have 90K input pixel values and 100K nodes in a hidden layer, we are looking at ~9 billion parameters, which is huge in terms of computation.

Convolution and pooling help in fetching a flatten layer that has a much smaller representation than the original image.

Finally, the last part of the classification is similar to the way we classified images in *Chapter 3*, where we had a hidden layer and then obtained the output layer.

How convolution and pooling help in image translation

When we perform pooling, we can consider the output of the operation as an abstraction of a region (a small patch). This phenomenon comes in handy, especially when images are being translated.

Think of a scenario where an image is translated by 1 pixel to the left. Once we perform convolution, activation, and pooling on top of it, we'll have reduced the dimension of the image (due to pooling), which means that a fewer number of pixels store the majority of the information from the original image. Moreover, given that pooling stores information of a region (patch), the information within a pixel of the pooled image would not vary, even if the original image is translated by 1 unit. This is because the maximum value of that region is likely to get captured in the pooled image.

Convolution and pooling can also help us with the **receptive field**. To understand the receptive field, let's imagine a scenario where we perform a convolution + pooling operation twice on an image that is 100 x 100 in shape. The output at the end of the two convolution pooling operations is of the shape 25 x 25 (if the convolution operation was done with padding). Each cell in the 25 x 25 output now corresponds to a larger 4 x 4 portion of the original image. Thus, because of the convolution and pooling operations, each cell in the resulting image contains key information within a patch of the original image.

Now that we have learned about the core components of a CNN, let's apply them all to a toy example to understand how they work together.

Implementing a CNN

A CNN is one of the foundational blocks of computer vision techniques, and it is important for you to have a solid understanding of how they work. While we already know that a CNN constitutes convolution, pooling, flattening, and then the final classification layer, in this section, we

will understand the various operations that occur during the forward pass of a CNN through code.

To gain a solid understanding of this, first, we will build a CNN architecture on a toy example using PyTorch and then match the output by building the feed-forward propagation from scratch in Python. The CNN architecture will differ from the neural network architecture that we built in the previous chapter in that a CNN constitutes the following in addition to what a typical vanilla deep neural network would have:

- Convolution operation
- Pooling operation
- Flatten layer

In the following code, we will build a CNN model on a toy dataset, as follows:

The following code can be found in the `CNN_working_details.ipynb` file located in the `Chapter04` folder on GitHub at **https://bit.ly/mcvp-2e**.

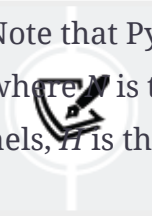
1. First, we need to import the relevant libraries:

```
import torch
from torch import nn
from torch.utils.data import TensorDataset, Dataset, DataLoader
from torch.optim import SGD, Adam
device = 'cuda' if torch.cuda.is_available() else 'cpu'
from torchvision import datasets
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

2. Then, we need to create the dataset using the following steps:

```
X_train = torch.tensor([[[[1,2,3,4],[2,3,4,5],
                          [5,6,7,8],[1,3,4,5]]],
```

```
                        [[[-1,2,3,-4],[2,-3,4,5],
            [-5,6,-7,8],[-1,-3,-4,-5]]]]).to(device).float()
X_train /= 8
y_train = torch.tensor([0,1]).to(device).float()
```



Note that PyTorch expects the input to be of the shape **N x C x H x W**, where *N* is the number (batch size) of images, *C* is the number of channels, *H* is the height, and *W* is the width of the image.

Here, we are scaling the input dataset so that it has a range between -1 and +1 by dividing the input data by the maximum input value – that is, 8. The shape of the input dataset is (2,1,4,4) since there are two data points, where each is 4 x 4 in shape and has 1 channel.

3. Define the model architecture:

```
def get_model():
    model = nn.Sequential(
                nn.Conv2d(1, 1, kernel_size=3),
                nn.MaxPool2d(2),
                nn.ReLU(),
                nn.Flatten(),
                nn.Linear(1, 1),
                nn.Sigmoid(),
            ).to(device)
    loss_fn = nn.BCELoss()
    optimizer = Adam(model.parameters(), lr=1e-3)
    return model, loss_fn, optimizer
```

Note that in the preceding model, we are specifying that there is 1 channel in the input and that we are extracting 1 channel from the output post-convolution (that is, we have 1 filter with a size of 3 x 3) using the `nn.Conv2d` method.

After this, we perform max pooling using `nn.MaxPool2d` and ReLU activation (using `nn.Relu()`) prior to flattening and connecting to the final layer, which has one output per data point.

Furthermore, note that the loss function is binary cross-entropy loss (`nn.BCELoss()`) since the output is from a binary class. We are also specifying that the optimization will be done using the Adam optimizer with a learning rate of 0.001.

4. Summarize the architecture of the model using the `summary` method that's available in the `torch_summary` package post fetching our `model`, loss function (`loss_fn`), and `optimizer` by calling the `get_model` function:

```
!pip install torch_summary
from torchsummary import summary
model, loss_fn, optimizer = get_model()
summary(model, X_train);
```

The preceding code results in the following output:

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1               [-1, 1, 2, 2]              10
         MaxPool2d-2               [-1, 1, 1, 1]               0
              ReLU-3               [-1, 1, 1, 1]               0
           Flatten-4                     [-1, 1]               0
            Linear-5                     [-1, 1]               2
           Sigmoid-6                     [-1, 1]               0
================================================================
Total params: 12
Trainable params: 12
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.00
Forward/backward pass size (MB): 0.00
Params size (MB): 0.00
Estimated Total Size (MB): 0.00
----------------------------------------------------------------
```

Figure 4.15: Summary of model architecture

Let's understand the reason why each layer contains as many parameters. The arguments of the `Conv2d` class are as follows:

```
help(nn.Conv2d)
```

```
Args:
    in_channels (int): Number of channels in the input image
    out_channels (int): Number of channels produced by the convolution
    kernel_size (int or tuple): Size of the convolving kernel
    stride (int or tuple, optional): Stride of the convolution. Default: 1
    padding (int or tuple, optional): Zero-padding added to both sides of the input. Default: 0
    padding_mode (string, optional). Accepted values `zeros` and `circular` Default: `zeros`
    dilation (int or tuple, optional): Spacing between kernel elements. Default: 1
    groups (int, optional): Number of blocked connections from input channels to output channels. Default: 1
    bias (bool, optional): If ``True``, adds a learnable bias to the output. Default: ``True``
```

Figure 4.16: Arguments within Conv2d

In the preceding case, we are specifying that the size of the convolving kernel (`kernel_size`) is 3 and that the number of `out_channels` is 1 (essentially, the number of filters is 1), where the number of initial (input) channels is 1. Thus, for each input image, we are convolving a filter of shape 3 x 3 on a shape of 1 x 4 x 4, which results in an output of the shape 1 x 2 x 2. There are 10 parameters since we are learning the nine weight parameters (3 x 3) and the one bias of the convolving kernel. For the `MaxPool2d`, ReLU, and flatten layers, there are no parameters as these are operations that are performed on top of the output of the convolution layer; no weights or biases are involved.

The linear layer has two parameters – one weight and one bias – which means there's a total of 12 parameters (10 from the convolution operation and two from the linear layer).

5. Train the model using the same model training code we used in *Chapter 3*, where we defined the function that will train on batches of data (`train_batch`). Then, fetch the `DataLoader` and train it on batches of data over 2,000 epochs (we're only using 2,000 because this is a small toy dataset), as follows:
    1. Define the function that will train on batches of data (`train_batch`):

```
def train_batch(x, y, model, opt, loss_fn):
    model.train()
    prediction = model(x)
    batch_loss = loss_fn(prediction.squeeze(0), y)
    batch_loss.backward()
    optimizer.step()
```

```
    optimizer.zero_grad()
    return batch_loss.item()
```

2. Define the training DataLoader by specifying the dataset using the `TensorDataset` method and then loading it using `DataLoader`:

```
trn_dl = DataLoader(TensorDataset(X_train, y_train))
```

   Given that we are not modifying the input data by a lot, we won't be building a class separately, but instead, leveraging the `TensorDataset` method directly, which provides an object that corresponds to the input data.
3. Train the model over 2,000 epochs:

```
for epoch in range(2000):
    for ix, batch in enumerate(iter(trn_dl)):
        x, y = batch
        batch_loss = train_batch(x, y, model,optimizer, loss_fn)
```

With the preceding code, we have trained the CNN model on our toy dataset.

6. Perform a forward pass on top of the first data point:

```
model(X_train[:1])
```

The output of the preceding code is `0.1625`.

Note that you might have a different output value owing to a different random weight initialization when you execute the preceding code.

Using the `CNN from scratch in Python.pdf` file in the GitHub repository, we can learn how forward propagation in CNNs works from scratch and replicate the output of 0.1625 on the first data point.

In the next section, we'll apply this to the Fashion-MNIST dataset and see how it fares on translated images.

Classifying images using deep CNNs

So far, we have seen that the traditional neural network predicts incorrectly for translated images. This needs to be addressed because, in real-world scenarios, various augmentations will need to be applied, such as translation and rotation, that were not seen during the training phase. In this section, we will understand how CNNs address the problem of incorrect predictions when image translation happens on images in the Fashion-MNIST dataset.

The preprocessing portion of the Fashion-MNIST dataset remains the same as in the previous chapter, except when we reshape (`.view`) the input data, where instead of flattening the input to 28 x 28 = 784 dimensions, we reshape the input to a shape of (1,28,28) for each image (remember, channels are to be specified first, followed by their height and width, in PyTorch):

The following code can be found in the `CNN_on_FashionMNIST.ipynb` file located in the `Chapter04` folder on GitHub at **https://bit.ly/mcvp-2e**.

1. Import the necessary packages:

```
from torchvision import datasets
from torch.utils.data import Dataset, DataLoader
import torch
import torch.nn as nn
device = "cuda" if torch.cuda.is_available() else "cpu"
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
data_folder = '~/data/FMNIST' # This can be any directory you
# want to download FMNIST to
fmnist = datasets.FashionMNIST(data_folder,download=True, train=True)
```

```
tr_images = fmnist.data
tr_targets = fmnist.targets
```

2. The Fashion-MNIST dataset class is defined as follows. Remember, the `Dataset` object will **always** need the `__init__`, `__getitem__`, and `__len__` methods we've defined:

```
class FMNISTDataset(Dataset):
    def __init__(self, x, y):
        x = x.float()/255
        x = x.view(-1,1,28,28)
        self.x, self.y = x, y
    def __getitem__(self, ix):
        x, y = self.x[ix], self.y[ix]
        return x.to(device), y.to(device)
    def __len__(self):
        return len(self.x)
```

The line of code in bold is where we are reshaping each input image (differently from what we did in the previous chapter) since we are providing data to a CNN that expects each input to have a shape of batch size x channels x height x width.

3. The CNN model architecture is defined as follows:

```
from torch.optim import SGD, Adam
def get_model():
    model = nn.Sequential(
                nn.Conv2d(1, 64, kernel_size=3),
                nn.MaxPool2d(2),
                nn.ReLU(),
                nn.Conv2d(64, 128, kernel_size=3),
                nn.MaxPool2d(2),
                nn.ReLU(),
                nn.Flatten(),
                nn.Linear(3200, 256),
                nn.ReLU(),
                nn.Linear(256, 10)
            ).to(device)
    loss_fn = nn.CrossEntropyLoss()
```

```
    optimizer = Adam(model.parameters(), lr=1e-3)
    return model, loss_fn, optimizer
```

4. A summary of the model can be created using the following code:

```
from torchsummary import summary
model, loss_fn, optimizer = get_model()
summary(model, torch.zeros(1,1,28,28));
```

This results in the following output:

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 64, 26, 26]             640
         MaxPool2d-2           [-1, 64, 13, 13]               0
              ReLU-3           [-1, 64, 13, 13]               0
            Conv2d-4          [-1, 128, 11, 11]          73,856
         MaxPool2d-5            [-1, 128, 5, 5]               0
              ReLU-6            [-1, 128, 5, 5]               0
           Flatten-7                 [-1, 3200]               0
            Linear-8                  [-1, 256]         819,456
              ReLU-9                  [-1, 256]               0
           Linear-10                   [-1, 10]           2,570
================================================================
Total params: 896,522
Trainable params: 896,522
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.00
Forward/backward pass size (MB): 0.69
Params size (MB): 3.42
Estimated Total Size (MB): 4.11
----------------------------------------------------------------
```

Figure 4.17: Summary of model architecture

To solidify our understanding of CNNs, let's understand the reason why the numbers of parameters have been set the way they have in the preceding output:

- **Layer 1**: Given that there are 64 filters with a kernel size of 3, we have 64 x 3 x 3 weights and 64 x 1 biases, resulting in a total of 640 parameters.

- **Layer 4**: Given that there are 128 filters with a kernel size of 3, we have 128 x 64 x 3 x 3 weights and 128 x 1 biases, resulting in a total of 73,856 parameters.
- **Layer 8**: Given that a layer with 3,200 nodes is getting connected to another layer with 256 nodes, we have a total of 3,200 x 256 weights and 256 biases, resulting in a total of 819,456 parameters.
- **Layer 10**: Given that a layer with 256 nodes is getting connected to a layer with 10 nodes, we have a total of 256 x 10 weights and 10 biases, resulting in a total of 2,570 parameters.

Now, we train the model, just like we trained it in the previous chapter.

The full code is available in this book's GitHub repository: `https://bit.ly/mcvp-2e`.

Once the model has been trained, you'll notice that the variation of accuracy and loss over the training and test datasets is as follows:

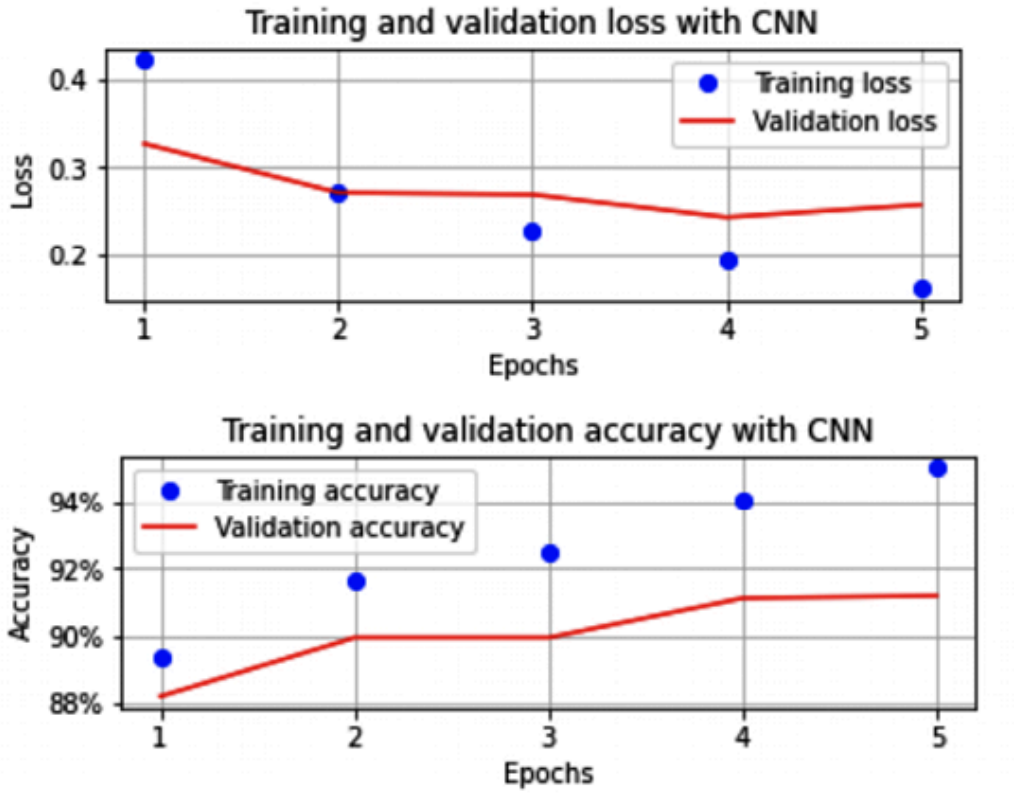


Figure 4.18: Training and validation loss and accuracy over increasing epochs

Note that in the preceding scenario, the accuracy of the validation dataset is ~92% within the first five epochs, which is already better than the accuracy we saw across various techniques in the previous chapter, even without additional regularization.

Let's translate the image and predict the class of translated images:

1. Translate the image between -5 pixels to +5 pixels and predict its class:

```
preds = []
ix = 24300
for px in range(-5,6):
    img = tr_images[ix]/255.
    img = img.view(28, 28)
    img2 = np.roll(img, px, axis=1)
    plt.imshow(img2)
    plt.show()
    img3 = torch.Tensor(img2).view(-1,1,28,28).to(device)
    np_output = model(img3).cpu().detach().numpy()
    preds.append(np.exp(np_output)/np.sum(np.exp(np_output)))
```

In the preceding code, we reshaped the image (`img3`) so it has a shape of `(-1,1,28,28)` , which will enable us to pass the image to a CNN model.

2. Plot the probability of the classes across various translations:

```
import seaborn as sns
fig, ax = plt.subplots(1,1, figsize=(12,10))
plt.title('Probability of each class for \
various translations')
sns.heatmap(np.array(preds).reshape(11,10), annot=True,
            ax=ax,fmt='.2f', xticklabels=fmnist.classes,
            yticklabels=[str(i)+str(' pixels') \
            for i in range(-5,6)], cmap='gray')
```

The preceding code results in the following output:

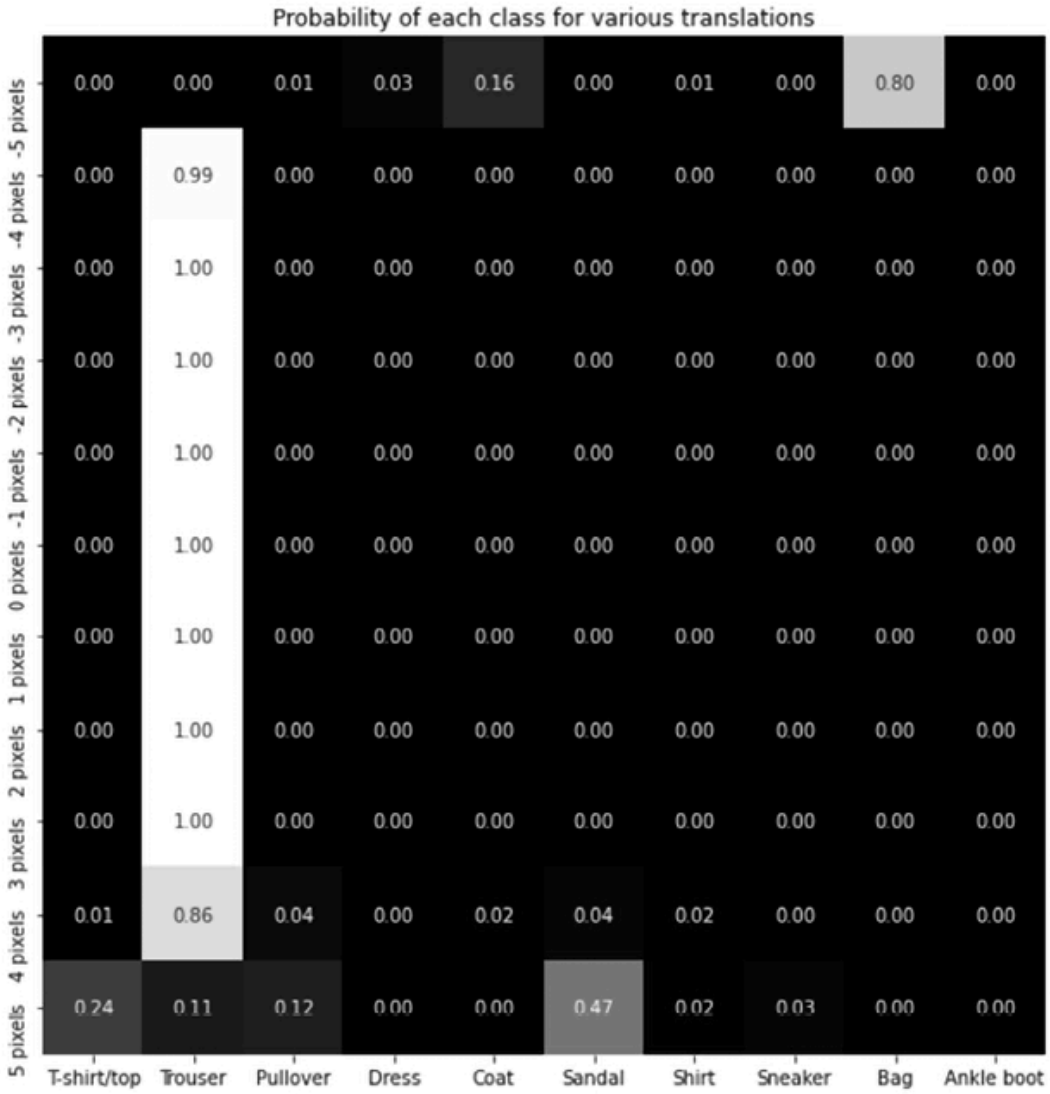


Figure 4.19: Probability of each class with varying translation

Note that in this scenario, even when the image was translated by 4 pixels, the prediction was correct, while in the scenario where we did not use a CNN, the prediction was incorrect when the image was translated by 4 pixels. Furthermore, when the image was translated by 5 pixels, the probability of **Trouser** dropped considerably.

As we can see, while CNNs help in addressing the challenge of image translation, they don't solve the problem at hand completely. We will learn how to address such a scenario by leveraging data augmentation alongside CNNs.

Given the different techniques that can be leveraged for data augmentation, we have provided exhaustive information on data augmentation in the GitHub repository in the `implementing data augmentation.pdf` file within the `Chapter04` folder.

Visualizing the outcome of feature learning

So far, we have learned how CNNs help us classify images, even when the objects in the images have been translated. We have also learned that filters play a key role in learning the features of an image, which, in turn, helps in classifying the image into the right class. However, we haven't mentioned what the filters learn that makes them powerful. In this section, we will learn about what these filters learn that enables CNNs to classify an image correctly by classifying a dataset that contains images of Xs and Os. We will also examine the fully connected layer (flatten layer) to understand what their activations look like.

Let's take a look at what the filters learn:

The following code can be found in the `Visualizing_the_features'_learning.ipynb` file located in the `Chapter04` folder on GitHub at **https://bit.ly/mcvp-2e**.

1. Download the dataset:

```
!wget https://www.dropbox.com/s/5jh4hpuk2gcxaaq/all.zip
!unzip all.zip
```

Note that the images in the folder are named as follows:

```
all/o@InterconnectedDemo-Bold@IttL47.png
all/o@Refresh-Regular@LX2MG4.png
all/x@CallistaOllander@7EWgpq.png
all/x@ChristmasSeason@xZ7mjB.png
```

Figure 4.20: Naming convention of images

The class of an image can be obtained from the image's name, where the first character of the image's name specifies the class the image belongs to.

2. Import the required modules:

```
import torch
from torch import nn
from torch.utils.data import TensorDataset,Dataset,DataLoader
from torch.optim import SGD, Adam
device = 'cuda' if torch.cuda.is_available() else 'cpu'
from torchvision import datasets
import numpy as np, cv2
import matplotlib.pyplot as plt
%matplotlib inline
from glob import glob
from imgaug import augmenters as iaa
```

3. Define a class that fetches data. Also, ensure that the images have been resized to a shape of 28 x 28, batches have been shaped with three channels, and that the dependent variable is fetched as a numeric value. We'll do this in the following code, one step at a time:
   1. Define the image augmented method, which resizes the image to a shape of 28 x 28:

```
tfm = iaa.Sequential(iaa.Resize(28))
```

   2. Define a class that takes the folder path as input and loops through the files in that path in the `__init__` method:

```
class XO(Dataset):
    def __init__(self, folder):
        self.files = glob(folder)
```

3. Define the `__len__` method, which returns the lengths of the files that are to be considered:

```
    def __len__(self): return len(self.files)
```

4. Define the `__getitem__` method, which we use to fetch an index that returns the file present at that index, read the file, and then perform augmentation on the image. We have not used `collate_fn` here because this is a small dataset and it wouldn't affect the training time significantly:

```
    def __getitem__(self, ix):
        f = self.files[ix]
        im = tfm.augment_image(cv2.imread(f)[:,:,0])
```

5. Given that each image is of the shape 28 x 28, we'll now create a dummy channel dimension at the beginning of the shape – that is, before the height and width of an image:

```
        im = im[None]
```

6. Now, we can assign the class of each image based on the character post `'/'` and prior to `'@'` in the filename:

```
        cl = f.split('/')[-1].split('@')[0] == 'x'
```

7. Finally, we return the image and the corresponding class:

```
        return torch.tensor(1 - im/255).to(device).float(),
                    torch.tensor([cl]).float().to(device)
```

4. Inspect a sample of the images you've obtained. In the following code, we're extracting the images and their corresponding classes by fetching data from the class we defined previously:

```
data = XO('/content/all/*')
```

Now, we can plot a sample of the images from the dataset we've obtained:

```
R, C = 7,7
fig, ax = plt.subplots(R, C, figsize=(5,5))
for label_class, plot_row in enumerate(ax):
    for plot_cell in plot_row:
        plot_cell.grid(False); plot_cell.axis('off')
        ix = np.random.choice(1000)
        im, label = data[ix]
        print()
        plot_cell.imshow(im[0].cpu(), cmap='gray')
plt.tight_layout()
```

The preceding code results in the following output:

Figure 4.21: Sample images

5. Define the model architecture, loss function, and the optimizer:

```
from torch.optim import SGD, Adam
def get_model():
    model = nn.Sequential(
                nn.Conv2d(1, 64, kernel_size=3),
                nn.MaxPool2d(2),
                nn.ReLU(),
                nn.Conv2d(64, 128, kernel_size=3),
                nn.MaxPool2d(2),
                nn.ReLU(),
                nn.Flatten(),
                nn.Linear(3200, 256),
                nn.ReLU(),
                nn.Linear(256, 1),
                nn.Sigmoid()
            ).to(device)
```

```
    loss_fn = nn.BCELoss()
    optimizer = Adam(model.parameters(), lr=1e-3)
    return model, loss_fn, optimizer
```

Note that the loss function is binary cross-entropy loss (`nn.BCELoss()`) since the output provided is from a binary class. A summary of the preceding model can be obtained as follows:

```
!pip install torch_summary
from torchsummary import summary
model, loss_fn, optimizer = get_model()
summary(model, torch.zeros(1,1,28,28));
```

This results in the following output:

Figure 4.22: Summary of model architecture

6. Define a function for training on batches that takes images and their classes as input and returns their loss values and accuracy after back-propagation has been performed on top of the given batch of data:

```
def train_batch(x, y, model, opt, loss_fn):
    model.train()
    prediction = model(x)
    is_correct = (prediction > 0.5) == y
    batch_loss = loss_fn(prediction, y)
    batch_loss.backward()
    optimizer.step()
    optimizer.zero_grad()
    return batch_loss.item(), is_correct[0]
```

7. Define a `DataLoader` where the input is the `Dataset` class:

```
trn_dl = DataLoader(XO('/content/all/*'),batch_size=32, drop_last=True)
```

8. Initialize the model:

```
model, loss_fn, optimizer = get_model()
```

9. Train the model over `5` epochs:

```
for epoch in range(5):
    for ix, batch in enumerate(iter(trn_dl)):
        x, y = batch
        batch_loss = train_batch(x, y, model,optimizer, loss_fn)
```

10. Fetch an image to check what the filters learn about the image:

```
im, c = trn_dl.dataset[2]
plt.imshow(im[0].cpu())
plt.show()
```

This results in the following output:

Figure 4.23: Sample image

11. Pass the image through the trained model and fetch the output of the first layer. Then, store it in the `intermediate_output` variable:

```
first_layer = nn.Sequential(*list(model.children())[:1])
intermediate_output = first_layer(im[None])[0].detach()
```

12. Plot the output of the 64 filters. Each channel in `intermediate_output` is the output of the convolution for each filter:

```
fig, ax = plt.subplots(8, 8, figsize=(10,10))
for ix, axis in enumerate(ax.flat):
    axis.set_title('Filter: '+str(ix))
    axis.imshow(intermediate_output[ix].cpu())
plt.tight_layout()
plt.show()
```

This results in the following output:

Figure 4.24: Activations of the 64 filters

Notice that certain filters, such as filters 0, 4, 6, and 7, learn about the edges present in the network, while other filters, such as filter 54, learn to invert the image.

13. Pass multiple O images and inspect the output of the fourth filter across the images (we are only using the fourth filter for illustration purposes; you can choose a different filter if you wish):
    1. Fetch multiple O images from the data:

```
x, y = next(iter(trn_dl))
```

```
x2 = x[y==0]
```

2. Reshape `x2` so that it has a proper input shape for a CNN model – that is, batch size x channels x height x width:

```
x2 = x2.view(-1,1,28,28)
```

3. Define a variable that stores the model until the first layer:

```
first_layer = nn.Sequential(*list(model.children())[:1])
```

4. Extract the output of passing the O images (`x2`) through the model until the first layer (`first_layer`), as defined previously:

```
first_layer_output = first_layer(x2).detach()
```

14. Plot the output of passing multiple images through the `first_layer` model:

```
n = 4
fig, ax = plt.subplots(n, n, figsize=(10,10))
for ix, axis in enumerate(ax.flat):
    axis.imshow(first_layer_output[ix,4,:,:].cpu())
    axis.set_title(str(ix))
plt.tight_layout()
plt.show()
```

The preceding code results in the following output:

Figure 4.25: Activations of the fourth filter when multiple O images are passed

Note that the behavior of a given filter (in this case, the fourth filter of

the first layer) has remained consistent across images.

15. Now, let's create another model that extracts layers until the second convolution layer (that is, until the four layers defined in the preceding model) and then extracts the output of passing the original O image. We will then plot the output of convolving the filters in the second layer with the input O image:

```
second_layer = nn.Sequential(*list(model.children())[:4])
second_intermediate_output=second_layer(im[None])[0].detach()
```

Plot the output of convolving the filters with the respective image:

```
fig, ax = plt.subplots(11, 11, figsize=(10,10))
for ix, axis in enumerate(ax.flat):
    axis.imshow(second_intermediate_output[ix].cpu())
    axis.set_title(str(ix))
plt.tight_layout()
plt.show()
```

The preceding code results in the following output:

Figure 4.26: Activations of the 128 filters in the second convolution layer

Now, let's use the 34th filter's output in the preceding image as an example. When we pass multiple O images through filter 34, we should see similar activations across images. Let's test this, as follows:

```
second_layer = nn.Sequential(*list(model.children())[:4])
second_intermediate_output = second_layer(x2).detach()
fig, ax = plt.subplots(4, 4, figsize=(10,10))
for ix, axis in enumerate(ax.flat):
    axis.imshow(second_intermediate_output[ix,34,:,:].cpu())
    axis.set_title(str(ix))
```

```
plt.tight_layout()
plt.show()
```

The preceding code results in the following output:

Figure 4.27: Activations of the 34th filter when multiple O images are passed

Note that, even here, the activations of the 34th filter on different images are similar in that the left half of O was activating the filter.

16. Plot the activations of a fully connected layer, as follows:
    1. First, fetch a larger sample of images:

```
custom_dl= DataLoader(XO('/content/all/*'),batch_size=2498, drop_last=True
```

    2. Next, choose only the O images from the dataset and then reshape them so that they can be passed as input to our CNN model:

```
x, y = next(iter(custom_dl))
x2 = x[y==0]
x2 = x2.view(len(x2),1,28,28)
```

    3. Fetch the flatten (fully connected) layer and pass the preceding images through the model until they reach the flatten layer:

```
flatten_layer = nn.Sequential(*list(model.children())[:7])
flatten_layer_output = flatten_layer(x2).detach()
```

    4. Plot the flatten layer:

```
plt.figure(figsize=(100,10))
plt.imshow(flatten_layer_output.cpu())
```

The preceding code results in the following output:

Figure 4.28: Activations of the fully connected layer

Note that the shape of the output is 1,245 x 3,200 since there are 1,245 **O** images in our dataset and there are 3,200 dimensions for each image in the flatten layer.

It's also interesting to note that certain values in the fully connected layer are highlighted when the input is **O** (here, we can see white lines, where each dot represents an activation value greater than zero).

Note that the model has learned to bring some structure to the fully connected layer, even though the input images – while all belonging to the same class – differ in style considerably.

Now that we have learned how CNNs work and how filters aid in this process, we will apply this so that we can classify images of cats and dogs.

Building a CNN for classifying real-world images

So far, we have learned how to perform image classification on the Fashion-MNIST dataset. In this section, we'll do the same for a more real-world scenario, where the task is to classify images containing cats or dogs. We will also learn how the accuracy of the dataset varies when we change the number of images available for training.

We will be working on a dataset available in Kaggle at **https://www.kaggle.com/tongpython/cat-and-dog**:

The following code can be found in the `Cats_Vs_Dogs.ipynb` file located in the `Chapter04` folder on GitHub at **https://bit.ly/mcvp-2e**. Be sure to copy the URL from the

notebook on GitHub to avoid any issues while reproducing the results.

1. Import the necessary packages:

```
import torchvision
import torch.nn as nn
import torch
import torch.nn.functional as F
from torchvision import transforms,models,datasets
from PIL import Image
from torch import optim
device = 'cuda' if torch.cuda.is_available() else 'cpu'
import cv2, glob, numpy as np, pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
from glob import glob
!pip install torch_summary
```

2. Download the dataset, as follows:
    1. We must download the dataset that's available in the `colab` environment. First, however, we must upload our Kaggle authentication file:

```
!pip install -q aggle
from google.colab import files
files.upload()
```

You will have to upload your `kaggle.json` file for this step, which can be obtained from your Kaggle account. Details on how to obtain the `kaggle.json` file is provided in the associated notebook on GitHub at **https://bit.ly/mcvp-2e**.

2. Next, specify that we're moving to the Kaggle folder and copy the `kaggle.json` file to it:

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!ls ~/.kaggle
!chmod 600 /root/.kaggle/kaggle.json
```

3. Finally, download the cats and dogs dataset and unzip it:

```
!kaggle datasets download -d tongpython/cat-and-dog
!unzip cat-and-dog.zip
```

3. Provide the training and test dataset folders:

```
train_data_dir = '/content/training_set/training_set'
test_data_dir = '/content/test_set/test_set'
```

4. Build a class that fetches data from the preceding folders. Then, based on the directory the image corresponds to, provide a label of 1 for dog images and a label of 0 for cat images. Furthermore, ensure that the fetched image has been normalized to a scale between 0 and 1 and permute it so that channels are provided first (as PyTorch models expect to have channels specified first, before the height and width of the image) – performed as follows:
    1. Define the `__init__` method, which takes a folder as input and stores the file paths (image paths) corresponding to the images in the `cats` and `dogs` folders in separate objects, post concatenating the file paths into a single list:

```
from torch.utils.data import DataLoader, Dataset
class cats_dogs(Dataset):
    def __init__(self, folder):
        cats = glob(folder+'/cats/*.jpg')
        dogs = glob(folder+'/dogs/*.jpg')
        self.fpaths = cats + dogs
```

    2. Next, randomize the file paths and create target variables based on the folder corresponding to these file paths:

```
        from random import shuffle, seed; seed(10);
        shuffle(self.fpaths)
        self.targets=[fpath.split('/')[-1].startswith('dog') \
                      for fpath in self.fpaths] # dog=1
```

3. Define the `__len__` method, which corresponds to the `self` class:

```
    def __len__(self): return len(self.fpaths)
```

4. Define the `__getitem__` method, which we use to specify a random file path from the list of file paths, read the image, and resize all the images so that they're 224 x 224 in size. Given that our CNN expects the inputs from the channel to be specified first for each image, we will permute the resized image so that channels are provided first before we return the scaled image and the corresponding `target` value:

```
    def __getitem__(self, ix):
        f = self.fpaths[ix]
        target = self.targets[ix]
        im = (cv2.imread(f)[:,:,::-1])
        im = cv2.resize(im, (224,224))
        return torch.tensor(im/255).permute(2,0,1).to(device).float(),\
               torch.tensor([target]).float().to(device)
```

5. Inspect a random image:

```
data = cats_dogs(train_data_dir)
im, label = data[200]
```

We need to permute the image we've obtained to our channels last. This is because `matplotlib` expects an image to have the channels specified after the height and width of the image have been provided:

```
plt.imshow(im.permute(1,2,0).cpu())
print(label)
```

This results in the following output:

Figure 4.29: Sample dog image

6. Define a model, loss function, and optimizer, as follows:
    1. First, we must define the `conv_layer` function, where we perform convolution, ReLU activation, batch normalization, and max pooling in that order. This method will be reused in the final model, which we will define in the next step:

```
def conv_layer(ni,no,kernel_size,stride=1):
    return nn.Sequential(
        nn.Conv2d(ni, no, kernel_size, stride),
        nn.ReLU(),
        nn.BatchNorm2d(no),
        nn.MaxPool2d(2)
    )
```

    In the preceding code, we are taking the number of input channels (`ni`), number of output channels (`no`), `kernel_size`, and the `stride` of filters as input for the `conv_layer` function.
    2. Define the `get_model` function, which performs multiple convolutions and pooling operations (by calling the `conv_layer` method), flattens the output, and connects a hidden layer to it prior to connecting to the output layer:

```
def get_model():
    model = nn.Sequential(
              conv_layer(3, 64, 3),
              conv_layer(64, 512, 3),
              conv_layer(512, 512, 3),
              conv_layer(512, 512, 3),
              conv_layer(512, 512, 3),
              conv_layer(512, 512, 3),
              nn.Flatten(),
              nn.Linear(512, 1),
```

```
            nn.Sigmoid(),
        ).to(device)
    loss_fn = nn.BCELoss()
    optimizer=torch.optim.Adam(model.parameters(), lr= 1e-3)
    return model, loss_fn, optimizer
```

You can chain `nn.Sequential` inside `nn.Sequential` with as much depth as you want. In the preceding code, we used `conv_layer` as if it were any other `nn.Module` layer.

3. Now, we must call the `get_model` function to fetch the model, loss function (`loss_fn`), and `optimizer` and then summarize the model using the `summary` method that we imported from the `torchsummary` package:

```
from torchsummary import summary
model, loss_fn, optimizer = get_model()
summary(model, torch.zeros(1,3, 224, 224));
```

The preceding code results in the following output:

Figure 4.30: Summary of model architecture

7. Create the `get_data` function, which creates an object of the `cats_dogs` class and creates a `DataLoader` with a `batch_size` of 32 for both the training and validation folders:

```
def get_data():
    train = cats_dogs(train_data_dir)
    trn_dl = DataLoader(train, batch_size=32, shuffle=True,
                                            drop_last = True)
    val = cats_dogs(test_data_dir)
    val_dl = DataLoader(val,batch_size=32, shuffle=True, drop_last = True)
    return trn_dl, val_dl
```

In the preceding code, we are ignoring the last batch of data by specifying that `drop_last = True`. We're doing this because the last batch might not be the same size as the other batches.

8. Define the function that will train the model on a batch of data, as we've done in previous sections:

```
def train_batch(x, y, model, opt, loss_fn):
    model.train()
    prediction = model(x)
    batch_loss = loss_fn(prediction, y)
    batch_loss.backward()
    optimizer.step()
    optimizer.zero_grad()
    return batch_loss.item()
```

9. Define the functions for calculating accuracy and validation loss, as we've done in previous sections:
    1. Define the `accuracy` function:

```
@torch.no_grad()
def accuracy(x, y, model):
    prediction = model(x)
    is_correct = (prediction > 0.5) == y
    return is_correct.cpu().numpy().tolist()
```

    Note that the preceding code for accuracy calculation is different from the code in the Fashion-MNIST classification because the current model (cats versus dogs classification) is being built for binary classification, while the Fashion-MNIST model was built for multi-class classification.
    2. Define the validation loss calculation function:

```
@torch.no_grad()
def val_loss(x, y, model):
    prediction = model(x)
```

```
    val_loss = loss_fn(prediction, y)
    return val_loss.item()
```

10. Train the model for `5` epochs and check the accuracy of the test data at the end of each epoch, as we've done in previous sections:
    1. Define the model and fetch the required DataLoaders:

```
trn_dl, val_dl = get_data()
model, loss_fn, optimizer = get_model()
```

    2. Train the model over increasing epochs:

```
train_losses, train_accuracies = [], []
val_losses, val_accuracies = [], []
for epoch in range(5):
    train_epoch_losses, train_epoch_accuracies = [], []
    val_epoch_accuracies = []
    for ix, batch in enumerate(iter(trn_dl)):
        x, y = batch
        batch_loss = train_batch(x, y, model,optimizer, loss_fn)
        train_epoch_losses.append(batch_loss)
    train_epoch_loss = np.array(train_epoch_losses).mean()
    for ix, batch in enumerate(iter(trn_dl)):
        x, y = batch
        is_correct = accuracy(x, y, model)
        train_epoch_accuracies.extend(is_correct)
    train_epoch_accuracy = np.mean(train_epoch_accuracies)
    for ix, batch in enumerate(iter(val_dl)):
        x, y = batch
        val_is_correct = accuracy(x, y, model)
        val_epoch_accuracies.extend(val_is_correct)
    val_epoch_accuracy = np.mean(val_epoch_accuracies)
    train_losses.append(train_epoch_loss)
    train_accuracies.append(train_epoch_accuracy)
    val_accuracies.append(val_epoch_accuracy)
```

11. Plot the variation of the training and validation accuracies over increasing epochs:

```
epochs = np.arange(5)+1
import matplotlib.ticker as mtick
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
%matplotlib inline
plt.plot(epochs, train_accuracies, 'bo',
         label='Training accuracy')
plt.plot(epochs, val_accuracies, 'r',
         label='Validation accuracy')
plt.gca().xaxis.set_major_locator(mticker.MultipleLocator(1))
plt.title('Training and validation accuracy \
with 4K data points used for training')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.gca().set_yticklabels(['{:.0f}%'.format(x*100) \
                           for x in plt.gca().get_yticks()])
plt.legend()
plt.grid('off')
plt.show()
```

The preceding code results in the following output:

Figure 4.31: Training and validation accuracy over increasing epochs

Note that the classification accuracy at the end of `5` epochs is ~86%.

As we discussed in the previous chapter, batch normalization has a great impact on improving classification accuracy – check this out for yourself by training the model without batch normalization. Furthermore, the model can be trained without batch normalization if you use fewer parameters. You can do this by reducing the number of layers, increasing the stride, increasing the pooling, or resizing the image to a number that's lower than 224 x 224.

So far, the training we've done has been based on ~8K examples, where 4K examples have been from the `cat` class and the rest have been from the `dog` class. In the next section, we will learn about the impact that having a reduced number of training examples has on each class when it comes to the classification accuracy of the test dataset.

Impact on the number of images used for training

We know that, generally, the more training examples we use, the better our classification accuracy is. In this section, we will learn what impact using different numbers of available images has on training accuracy by artificially reducing the number of images available for training and then testing the model's accuracy when classifying the test dataset.

The following code can be found in the `Cats_Vs_Dogs.ipynb` file located in the `Chapter04` folder on GitHub at **https://bit.ly/mcvp-2e**. Given that the majority of the code that will be provided here is similar to what we have seen in the previous section, we have only provided the modified code for brevity. The respective notebook on GitHub will contain the full code.

Here, we only want to have 500 data points for each class in the training dataset. We can do this by limiting the number of files to only the first 500 image paths in each folder in the `__init__` method and ensuring that the rest remain as they were in the previous section:

```
    def __init__(self, folder):
        cats = glob(folder+'/cats/*.jpg')
        dogs = glob(folder+'/dogs/*.jpg')
        self.fpaths = cats[:500] + dogs[:500]
        from random import shuffle, seed; seed(10);
            shuffle(self.fpaths)
        self.targets = [fpath.split('/')[-1].startswith('dog') \
                        for fpath in self.fpaths]
```

In the preceding code, the only difference from the initialization we performed in the previous section is in `self.paths`, where we are now limiting the number of file paths to be considered in each folder to only the first 500.

Now, once we execute the rest of the code, as we did in the previous section, the accuracy of the model that's been built on 1,000 images (500 of each class) in the test dataset will be as follows:

Figure 4.32: Training and validation accuracy with 1K data points

We can see that because we had fewer examples of images in training, the accuracy of the model on the test dataset reduced considerably – that is, down to ~66%.

Now, let's see how the number of training data points impacts the accuracy of the test dataset by varying the number of available training examples that will be used to train the model (where we build a model for each scenario).

We'll use the same code we used for the 1K (500 per class) data point training example but will vary the number of available images (to 2K, 4K, and 8K total data points, respectively). For brevity, we will only look at the output of running the model on a varying number of images available for training. This results in the following output:

Figure 4.33: Training and validation accuracy with varying number of data points

As you can see, the more training data that's available, the higher the accuracy of the model on test data. However, we might not have a large enough amount of training data in every scenario that we encounter. The next chapter, which will cover transfer learning, will address this prob-

lem by walking you through various techniques you can use to attain high accuracy, even on a small amount of training data.

Summary

Traditional neural networks fail when new images that are very similar to previously seen images that have been translated are fed as input to the model. CNNs play a key role in addressing this shortcoming. This is enabled through the various mechanisms that are present in CNNs, including filters, strides, and pooling. Initially, we built a toy example to learn how CNNs work. Then, we learned how data augmentation helps in increasing the accuracy of the model by creating translated augmentations on top of the original image. After that, we learned about what different filters learn in the feature learning process so that we could implement a CNN to classify images.

Finally, we saw the impact that differing amounts of training data have on the accuracy of test data. Here, we saw that the more training data that is available, the better the accuracy of the test data. In the next chapter, we will learn how to leverage various transfer learning techniques to increase the accuracy of the test dataset, even when we have just a small amount of training data.

Questions

1. Why was the prediction on the translated image in the first section of the chapter low when using traditional neural networks?
2. How is convolution done?
3. How are optimal weight values in a filter identified?
4. How does the combination of convolution and pooling help in addressing the issue of image translation?
5. What do the convolution filters in layers closer to the input layer learn?
6. What functionality does pooling have that helps in building a model?
7. Why can’t we take an input image, flatten it just like we did on the Fashion-MNIST dataset, and train a model for real-world images?
8. How does data augmentation help in improving image translation?

9. In what scenario do we leverage `collate_fn` for dataloaders?
10. What impact does varying the number of training data points have on the classification accuracy of the validation dataset?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

https://packt.link/modcv